

Foundations of Artificial Intelligence - 2026.1

Fixed Size Decision Problems under Certainty

Felix Mohr

Universidad de La Sabana

Chía, February 4, 2026

Course Outline

1. Fixed Size Decision Problems
2. Constraint Networks
3. General Constructive Search and Its Application with Constraint Networks
4. Transformative Search - Genetic Algorithms

Course Outline

1. Fixed Size Decision Problems
2. Constraint Networks
3. General Constructive Search and Its Application with Constraint Networks
4. Transformative Search - Genetic Algorithms

Fixed Size Decision Problems

Often the number of decisions to be made is fixed.

We then say that we have **decision variables** $X_1, \dots, X_d$ with **domains** $D_1, \dots, D_d$.

This is called a **variable assignment problem**.

In this case and under certainty, it doesn't matter whether this is atomic or sequential decision making:

- For each product in the catalog, decide how many we purchase.
- Convert a todo-set into an optimal todo-list (like tour planning).

Fixed Size Decision Problems

Vectors and Assignment

Two essential ways of defining $\mathcal{X}$ are then:

- through **value tuples**: $\mathcal{X} \subseteq D_1 \times \dots \times D_d$.

Advantage: very efficient

Disadvantage: less flexible

- through **assignments**: $\mathcal{X} \subseteq \{(X_1, v_1), \dots, (X_d, v_d) \mid v_i \in D_i\}$.

Advantage: very flexible manipulations, specifically for partial assignments

Disadvantage: less efficient

Fixed Size Decision Problem

Assignment Sets

At least at the conceptual level, it is easier to work with assignments.

For further simplicity, we will define the set of all assignments with just one variable as

$$A = \cup_{i=1}^d \{(X_i, v) \mid v \in D_i\}$$

and the set of all syntactically possible (not necessarily legal) assignments as

$$\mathcal{A} = \{a \subset A \mid (X_i, u), (X_j, v) \in a \Rightarrow X_i \neq X_j\}$$

Finally, we denote as $\bar{\mathcal{A}} = \{a \in \mathcal{A} \mid \#a = d\}$ the set of **complete assignments**.

Fixed Size Decision Problems

Solution with Simple Constructive Search

Decision problems with a fixed number of decision variables are relatively easy to handle.

For Simple Constructive Search

- we could set $O = A$, i.e., each operation fixes exactly one variable value
- the search workspace would then be $\mathcal{W} = \{W \mid W \in O^* \wedge \text{set}(W) \in \mathcal{A}\}$
- the goal function is defined as $\star(W) = 1$ iff $\text{set}(W) \in \mathcal{X}$.

Here, $\text{set}(W) := \{o \mid o \in W\}$ casts the list W into a set with the same values.

One option for successors is: $\text{succ}(W) = \{W' \in \mathcal{W} \mid W = W'_{[: -1]}\}$

Fixed Size Decision Problems

Solution with Simple Transformative Search

For this problem class, it is also clear that transformational search can be easy:

We can just overwrite the variable values in an assignment with new values.

However, it can happen that such an overwriting makes that the new assignment, even if it has valid values for all variables, is not in $\mathcal{X}$ anymore.

Examples:

- If we have a fixed budget, we might set the number of purchases per item in a way that exceeds the budget, even if each item stays in its allowed range.
- If we change a tour in such a way that a location is now visited several times while another is not visited at all.

Course Outline

1. Fixed Size Decision Problems
2. Constraint Networks
3. General Constructive Search and Its Application with Constraint Networks
4. Transformative Search - Genetic Algorithms

Constraint Networks

Motivation

Consider a search space with

$$D_1 = D_2 = D_3 = \{1, 2, 3\}$$

and constraints $X_3 \leq X_1$ and $X_2 + X_3 = 5$. What happens if we assign $X_1 = 1$?

Simple Constructive Search is inefficient (wasted time) if it runs into pre-programmed dead-ends. We should eliminate dead-end-enforcing values from domains D_i .

Constraint Networks

Constraints

A **constraint** is a set $C \subseteq \mathcal{A}$ of assignments.

The membership $\mathbf{x} \in C$ means that assignment $\mathbf{x}$ is *legal* with respect to the constraint.

Observe that C may contain both complete and incomplete assignments.

An **indexed constraint** is a pair (J, C_J) , where C_J is a constraint and J are the variables whose values determine constraint membership.

Constraint Networks

Constraint Sets

We can characterize the search space $\mathcal{X}$ through a set of (maybe indexed) constraints

$$\mathcal{C} := \{(J_1, C_{J_1}), \dots, (J_m, C_{J_m})\}$$

If we have multiple constraints on the same set of variables J with associated sets $C_J^1, \dots, C_J^m$, we can merge these into a single constraint $(J, \cap_{j=1}^m C_J^j)$.

We can hence assume that $\mathcal{C}$ is a *function* with at most one constraint per variable set J .

We then get

$$\mathcal{X} = \bar{\mathcal{A}} \cap \bigcap_{(J, C_J) \in \mathcal{C}} C_J,$$

Constraint Networks

Different Constraint Sets Yield same Search Space

There are many ways to characterize the same search space $\mathcal{X}$ through constraints.

In particular, we can describe $\mathcal{X}$ in a single (indexed) constraint $(\{X_1, \dots, X_d\}, \tilde{\mathcal{X}})$ if

$$\tilde{\mathcal{X}} := \{\mathbf{x} \mid \mathbf{x} \in \mathcal{A} \wedge \mathbf{x} \text{ has a completion in } \mathcal{X}\}$$

But it is often more useful to *decompose* the definition of $\mathcal{X}$ into small constraints over few variables, because these can be checked in a modular way and when relevant.

Example: In Sudoku, we can use one constraint for each row, each of which only talks about 9 variables, and only needs to be checked if one of those changed.

Constraint Networks

All-Diff Constraint

One of the most important constraints is the All-Diff constraint.

For any set J of variables, it defines:

$$(J, \{\mathbf{x} \mid \mathbf{x}_J \text{ is injective}\}),$$

where $\mathbf{x}_J$ is projection of $\mathbf{x}$ to those entries with keys in J .

For example, the $\mathcal{X}$ in a given Sudoku can be described through 27 All-Diff constraints.

Constraint Networks

Implementation

In practice, a constraint is implemented in an *implicit* way through a function that

- receives a (possibly partial) assignment $\mathbf{x}$, e.g., as a dictionary, and
- returns true iff $\mathbf{x} \in C_J$.

If $\mathbf{x}$ does not define all variables in J , the constraint might still check whether there exists a completion of $\mathbf{x}$ towards J that is inside C_J .

Arc Consistency

Suppose that we can write the constraints that limit $\mathcal{X}$ as a set $\mathcal{C}$ of only **binary constraints** of the form $(\{X_i, X_j\}, C_{ij})$.

The set of constraints can then be drawn as a graph where

- variables are nodes and
- there is an edge between X_i and X_j if there is a constraint C_{ij} defined

The pair $(\mathcal{D}, \mathcal{C})$ is then called a **binary constraint net (BCN)**.

Arc Consistency

Binary Constraint Nets

Few constraints are natively binary, but some can be translated.

For example, the All-Diff constraint on variables J can be translated into $|J| \cdot (|J| - 1)/2$ binary (inequality) constraints.

Arc Consistency

A variable X_i is **arc consistent** with respect to X_j if for all $x_i \in D_i$ there is an $x_j \in D_j$ such that $(x_i, x_j) \in C_{\{i,j\}}$.

The *arc* $\{X_i, X_j\}$ is consistent if X_i is arc consistent with X_j and vice versa.

A constraint net is arc consistent if all of its edges are arc consistent.

Theorem: If a constraint net has a solution, it is arc consistent.

Algorithm REVISE

Require: Variables X_i, X_j with domains D_i and D_j , constraint $C_{\{i,j\}}$

Ensure: D'_i such that X_i is arc consistent with X_j

- 1: $D'_i \leftarrow \emptyset$
- 2: **for** $x_i \in D_i$ **do**
- 3: **if** $\exists x_j \in D_j : \{(X_i, x_i), (X_j, x_j)\} \in C_{\{i,j\}}$ **then**
- 4: $D'_i \leftarrow D'_i \cup \{x_i\}$
- 5: **end if**
- 6: **end for**
- 7: **return** D'_i

Algorithm AC1

Require: BCN $(\mathcal{D}, \mathcal{C})$

Ensure: arc consistent net that is equivalent to $(\mathcal{D}, \mathcal{C})$

- 1: $\mathcal{D}' \leftarrow \text{clone}(\mathcal{D})$
- 2: **repeat**
- 3: **for** binary constraint $((i, j), C_{ij}) \in \mathcal{C}$ **do**
- 4: $D_i \leftarrow \text{REVISE}(D_i, D_j, C_{\{i, j\}})$
- 5: $D_j \leftarrow \text{REVISE}(D_j, D_i, C_{\{i, j\}})$
- 6: **end for**
- 7: **until** No domain has changed in the last iteration
- 8: **return** BCN $(\mathcal{D}', \mathcal{C})$

The runtime complexity of AC1 is $\mathcal{O}(e \cdot n \cdot t^3)$, where e is the number of constraints.

Algorithm AC3

Require: BCN $(\mathcal{D}, \mathcal{C})$

Ensure: arc consistent net that is equivalent to the input

```
1: for binary constraint  $C_{\{X_i, X_j\}}$  do
2:    $queue \leftarrow queue \cup \{(X_i, X_j), (X_j, X_i)\}$ 
3: end for
4: while  $queue \neq \emptyset$  do
5:   poll  $(X_i, X_j)$  from  $queue$ 
6:    $D_i \leftarrow \text{REVISE}(X_i, X_j)$ 
7:   if REVISE changed  $D_i$  then  $queue \leftarrow queue \cup \{(X_k, X_i) \mid k \neq i, k \neq j\}$ 
8: end while
9: return BCN  $(\mathcal{D}', \mathcal{C})$ 
```

The runtime complexity of AC3 is $\mathcal{O}(et^3)$.

Course Outline

1. Fixed Size Decision Problems
2. Constraint Networks
3. General Constructive Search and Its Application with Constraint Networks
4. Transformative Search - Genetic Algorithms

Constraint Networks

Integrating AC into Simple Constructive Search

Using arc consistency in Constructive Search is easy if we have a BCN $(\mathcal{D}_0, \mathcal{C})$.

Define a helper function r that maps items $W \in \mathcal{W}$ to a reduced domain definition $\mathcal{D}' = r(W)$ by (i) setting $D'_i = \{v_i\}$ for variables fixed in W and (ii) then running AC3.

Then

- $\text{succ}(W)$ selects a $(X_i, D_i) \in \mathcal{D} = r(W)$ with $|D_i| > 1$, and, for each $x_i \in D_i$,
 - ▶ reduces domain $D'_i = \{x_i\}$, which defines $\mathcal{D}'$
 - ▶ obtains $\mathcal{D}'' = \text{AC3}((\mathcal{D}', \mathcal{C}))$
 - ▶ if $|D_i| \geq 1$ for all D_i in $\mathcal{D}''$, add $\{(X_j, v_j) \mid D_j = \{v_j\} \text{ in } \mathcal{D}'', X_j \text{ not fixed yet in } W\}$.
- $\star(W)$ checks whether $|D_i| = 1$ for all D_i in $\mathcal{D} = r(W)$.

So the operation set O now contains assignments of maybe multiple variables.

General Constructive Search

Motivation to not use Simple Constructive Search

The usage of *AC3* above is very inefficient since

- we run it twice for each workspace item
- domain reduction with *AC3* is monotonic, but we run it from scratch each time without saving results.

This suggests that, even though we are constructing assignments, the Simple Constructive Search is too limited.

General Constructive Search

Requirement Definition

We now require two things for each workspace item:

- provide the partial construction plan committed so far (we have that already)
- provide implications of this plan that facilitates computations

This is not compatible with the idea that workspace items are operation lists, but this restriction was already annoying before, when we wanted those to be sets instead of lists.

General Constructive Search

Algorithm Component Definition

An elegant way to solve these problems is to not work on O^* directly but arbitrary $\mathcal{W}$ with a restricted link to O^* . Allowing arbitrary $\mathcal{W}$, we then

- redefine $\text{succ} : \mathcal{W} \rightarrow 2^{\mathcal{W}}$,
- introduce a link function $\sigma : \mathcal{W} \rightarrow O^*$,
- explicitly require an initial workspace item $W_0 \in \mathcal{W}$, which before was $()$.

The search algorithm with this modification is **General Constructive Search**.

We usually require growing construction plans: $W' \in \text{succ}(W) \Rightarrow \sigma(W) \subset \sigma(W')$.

If succ and σ fulfill this, we get **General Monotone Constructive Search**.

General Constructive Search

Properties



This is a strict generalization of the Simple Constructive Search in which $\mathcal{W}$ is defined directly on O^* and $\sigma(W) = W$ is the identity function.

The fact that we can now choose $\mathcal{W}$ arbitrarily and not just lists of operations is an important and almost unlimited degree of freedom.

Without monotonicity, the search could remove or change operations. Monotonicity is not a property of the algorithm code but is (or not) implied by *succ* and σ .

Generic Decision Optimization under Certainty

General Constructive Search

Require: $(\langle W_0, succ, \star, \succ \rangle)$

```
1:  $OPEN \leftarrow \{W_0\}, W^* = \perp$ 
2: while  $OPEN \neq \emptyset$  and there is still time do
3:    $W \leftarrow \text{Remove-First}(OPEN)$ 
4:   for  $W' \in succ(W)$  do
5:     if  $\star(W')$  then
6:       if  $\succ$  is None then
7:         return  $W'$ 
8:       else if  $(W^* = \perp) \vee (W' \succ W^*)$  then
9:          $W^* \leftarrow W'$ 
10:      end if
11:    else
12:       $OPEN \leftarrow OPEN \cup \{W'\}$ 
13:    end if
14:  end for
15: end while
16: return  $W^*$ ;
```

General Constructive Search

Constraint Nets in General Constructive Search

Let $\mathcal{W}$ be the set of functions $\mathcal{D}$ from variables to reduced domains, i.e., $\mathcal{D}(X_i) \subseteq D_i$.

Then

- $W_0 := \mathcal{D}_1$ obtained from $AC3((\mathcal{D}_0, \mathcal{C}))$
- $\star(\mathcal{D}) := 1$ iff $|D_i| = 1$ for all $(X_i, D_i) \in \mathcal{D}$.
- $succ(\mathcal{D})$ selects a $(X_i, D_i) \in \mathcal{D}$ with $|D_i| > 1$, and, for each $x_i \in D_i$,
 - ▶ reduces domain $D'_i = \{x_i\}$, which defines $\mathcal{D}'$
 - ▶ obtains $\mathcal{D}'' = AC3((\mathcal{D}', \mathcal{C}))$
 - ▶ if $|D_i| \geq 1$ for all D_i in $\mathcal{D}''$, add a respective successor $\mathcal{D}''$.
- $\sigma(\mathcal{D})$ concatenates tuples (X_i, v_i) for those $(X_i, D_i) \in \mathcal{D}$ with $D_i = \{v_i\}$.

Summary of Constructive Search for Fixed Variable Size Problems

Method	O	$\mathcal{W}$	Improvement
Direct Simple Constructive Search	A	$\subset O^*$	-
Constraint Nets + Simple Constructive Search	$\mathcal{A}$	$\subset O^*$	dead-end detection
Constraint Nets + General Constructive Search	$\mathcal{A}$	$\mathcal{D}$	efficient usage of AC3

In the last row, the presence of σ generalizes the relationship between the construction operations O and $\mathcal{W}$, which was previously rigidly $\mathcal{W} \subseteq O^*$.

Course Outline

1. Fixed Size Decision Problems
2. Constraint Networks
3. General Constructive Search and Its Application with Constraint Networks
4. Transformative Search - Genetic Algorithms

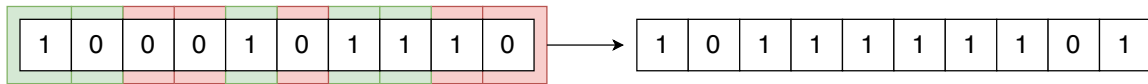
Transformative Search

The idea of **transformative search** is to take one or more *currently* known elements of $\mathcal{X}$ and *transform* them into other, hopefully better solutions.

The assumption here is that it is easy to

- create initial solutions from $\mathcal{X}$ and to
- move inside $\mathcal{X}$ via *manipulations* of candidate encodings.

Example: If $\mathcal{X} = \{0, 1\}^n$, we can simply flip bits to move inside $\mathcal{X}$:



Transformative Search

At an abstract level, all transformative algorithms have the same scheme:

- ① Create one or more initial solution(s) from $\mathcal{X}$.
- ② Iteratively
 - ① Compute transformed solutions in $\mathcal{X}$ from the current solution set.
 - ② Select a subset (typically best k) of the union of these transformed and the current solutions to become the new set of solutions.

The algorithms terminate if the transformations lead to solutions that are not or only marginally better than the current solutions (for some period of time).

Transformative Search

Encodings

For transformative searches, the encoding of search space elements is fundamental, because the movement inside $\mathcal{X}$ is defined in terms of operators on those encodings.

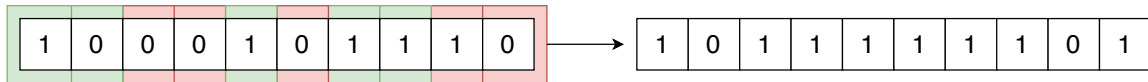
Typical encodings are

- **vectorial encodings**, e.g. $\{0, 1\}^n$ or Σ^n for some alphabet Σ and fixed n . Usually one may want to assume a certain independency of the different genes.
- **permutation encoding**, i.e. all possible ordering for some $n \in \mathbb{N}$
- **tree encoding** for more complex problems, e.g. flexible length chromosomes

Transformative Search

Typical Transformations: Flip-Bit

In this transformation, we have some vector $p \in [0, 1]^n$ that defines the probability for each variable to be changed. In the case of binary variables, the new value is unique:



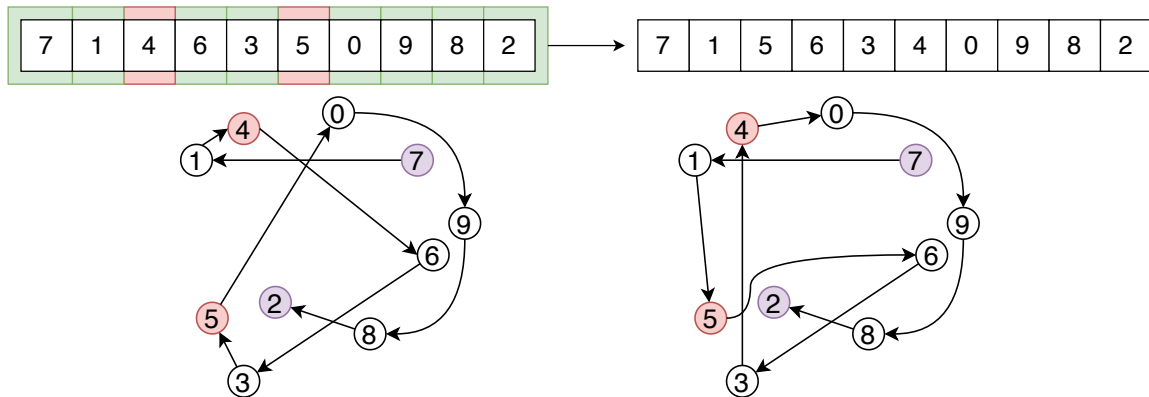
In the case of discrete variables, we need a distribution over the possible values (conditioned on the current value):



Transformative Search

Typical Transformations: Swap in Permutations

In case of permutation encodings, we can maintain validity (membership of $\mathcal{X}$) by exchanging two or more elements, i.e. to apply a permutation to the permutation.



Transformative Search

Random Restarts

Transformative searches often have difficulties to escape from **local optima**, and since the initial solutions are often random, their final output is subject to randomness as well.

It hence can make sense to restart the routine several times with different initializations (**random restarts**) and so increase the chance to find or approximate a local optimum.

Genetic Algorithms

Overview

Genetic Algorithms (GAs) or Evolutionary Algorithms (EAs) are transformative searches that *combine* currently considered solutions to derive new ones.

The combination of two or more parents into new **offsprings** is called a **crossover**.

In GAs, solutions are called **individuals**, and the current set of solutions is the **population**.

For the crossover, one needs to identify a collection of parent sets (subsets of population), each to which the crossover will be applied.

It is common to apply mutation to the offsprings before reducing the population.

Genetic Algorithms

Divide Crossover

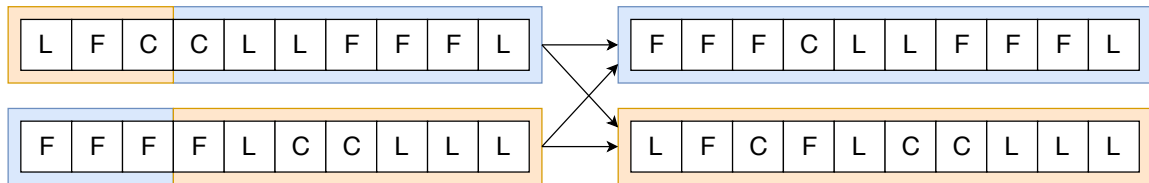
Combine

$$p = (p_1, \dots, p_n) \text{ and } q = (q_1, \dots, q_n)$$

into

$$p' = (p_1, \dots, p_l, q_{l+1}, \dots, q_n) \text{ and } q' = (q_1, \dots, q_l, p_{l+1}, \dots, p_n)$$

where $l \in \{1, \dots, n-1\}$ is chosen at random.



Genetic Algorithms

2-Point-Crossover

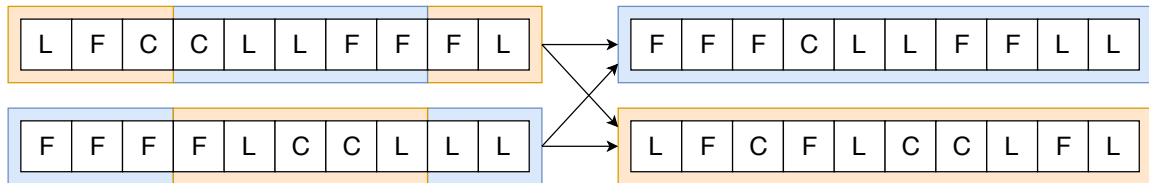
Combine

$$p = (p_1, \dots, p_n) \text{ and } q = (q_1, \dots, q_n)$$

into

$$p' = (p_1, \dots, p_l, q_{l+1}, \dots, q_r, p_{r+1}, \dots, p_n) \text{ and } q' = (q_1, \dots, q_l, p_{l+1}, \dots, p_r, q_{r+1}, \dots, q_r)$$

where $l, r \in \{1, \dots, n-1\}, l < r$ are chosen at random.



Genetic Algorithms

Uniform Crossover

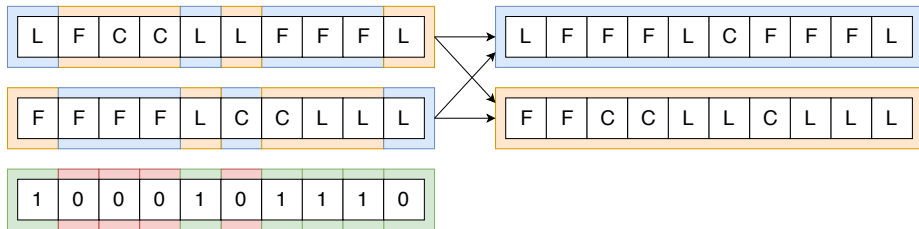
Combine

$$p = (p_1, \dots, p_n) \text{ and } q = (q_1, \dots, q_n)$$

into

$$p' = (p \wedge b) \vee (q \wedge \neg b) \text{ and } q' = (p \wedge \neg b) \vee (q \wedge b)$$

where $b \in \{0, 1\}^n$ is some random bit-mask.



Summary

Summarizing this class in one sentence, we have looked at variable assignment problems and learned that for those problems

- constraint nets can significantly improve runtimes of constructive search by detecting dead ends early, sometimes even making it obsolete altogether;
- full performance improvement requires an *implicit* representation of operations in constructive search instead of operation lists; and that
- we can use *genetic algorithms* as an algorithm class of transformative methods that *combines* two existing solutions instead of just modifying a single one.